

E-Learning JCB — Guion de Exposición Teórica (15 minutos)

~130 palabras/minuto · ~1.950 palabras · Solo exposición oral La demostración práctica (15 min) se realiza a continuación en vivo

ESTRUCTURA

Bloque	Tema	PDF	Tiempo
1	Introducción y contexto	Pág. 1–2	1 min
2	Mercado, oportunidad y modelo de negocio	Pág. 3–4	2 min
3	Viabilidad: técnica, económica y legal	Pág. 5	2 min
4	Arquitectura general del sistema	Pág. 6	1.5 min
5	La base de datos: MongoDB y modelos	Pág. 7	1.5 min
6	Autenticación y sistema de roles	Pág. 8	2 min
7	Los estados de Reflex: el backend	Pág. 9	2 min
8	Páginas, componentes y funcionalidades	Pág. 10	1 min
9	Conclusiones, ayudas y próximos pasos	Pág. 11–12	1 min

BLOQUE 1 — INTRODUCCIÓN Y CONTEXTO

🕒 1 minuto · ~130 palabras · Pág. 1 del PDF

Buenos días a todos.

Voy a presentaros **E-Learning JCB**, una plataforma de formación online construida íntegramente en Python. No hay ni una sola línea de JavaScript escrita manualmente.

Esto es posible gracias a **Reflex**, un framework que compila código Python en React + FastAPI automáticamente. Escribes Python puro; Reflex genera el frontend y el backend por debajo.

La plataforma **ya está desplegada en producción** en Reflex Cloud y accesible públicamente.

(Pág. 2 del PDF — métricas del proyecto)

Para tener la escala en mente:

- **39 archivos Python, 0 líneas de JavaScript manual**
- **30 páginas web, 33 rutas registradas**
- **10 clases de estado** gestionando toda la lógica
- **~18.000 líneas de código**

BLOQUE 2 — MERCADO, OPORTUNIDAD Y MODELO DE NEGOCIO

🕒 2 minutos · ~260 palabras · Pág. 3–4 del PDF · Módulo: EMPRESA

(Pág. 3 del PDF — mercado e-learning)

El e-learning es uno de los sectores con mayor crecimiento sostenido de la última década: mercado global de **\$315 mil millones** en 2023, proyección de **\$457 mil millones** para 2026 a un CAGR del **13,7%**. [Retorno anual](#)

He identificado tres segmentos con potencial real:

Formación técnica: 120.000 profesionales IT en España. Ticket medio 49€–199€. Proyección conservadora Año 1: **13.860€** con 100 estudiantes.

[Busines2Busines](#)

Corporativo B2B: 2,9 millones de PYMEs. Con solo 5 empresas cliente: **12.000€** al año.

Marketplace de instructores: 20 instructores, 50 ventas/mes, comisión 25% → **14.850€** al año.

Los competidores directos —Udemy, Coursera, Domestika, Platzi— dejan desatendido el nicho de **formación técnica accesible y de calidad para el mercado español**. Ese es nuestro hueco.

(Pág. 4 del PDF — modelo de negocio)

El modelo de negocio tiene tres patas:

1. **Freemium:** cursos básicos gratuitos, premium de 49€ a 99€.
2. **Comisiones:** 25–30% por venta de instructor.
3. **Suscripciones corporativas:** desde 200€/mes.

El **punto de equilibrio son solo 24 ventas al año** —2 cursos al mes—, un umbral altamente accesible.

BLOQUE 3 — VIABILIDAD: TÉCNICA, ECONÓMICA Y LEGAL

🕒 2 minutos · ~260 palabras · Pág. 5 del PDF · Módulo: FOL + EMPRESA

(Pág. 5 del PDF — viabilidad)

Forma jurídica: autónomo

Se ha optado por **trabajador autónomo** por tres razones: coste de constitución cero, gestión simplificada y **tarifa plana de 80€/mes** los primeros doce meses —ahorro de 2.676€—. Si el negocio supera los 40.000€ de beneficio, el cambio a SL es más eficiente fiscalmente.

Obligaciones fiscales

- **IVA 21%:** Modelo 303 trimestral + Modelo 390 anual.
 - **IRPF:** Modelo 130 trimestral, estimación directa simplificada.
 - **RETA:** 80€/mes Año 1; cuota por rendimientos netos a partir del Año 2.
- Régimen Especial de Trabajadores Autónomos

Marco legal

- Reglamento General de Protección de Datos
- **RGPD:** datos en MongoDB Atlas con cifrado AES-256 (reposo) y TLS (tránsito), política de privacidad y gestión de cookies.
 - **LSSI:** precios visibles, contratación electrónica documentada.
 - **Propiedad intelectual:** código bajo **licencia MIT**, dependencias bajo Apache 2.0/PSF.
- Ley de Servicios de la Sociedad de la Información

Resumen económico

Concepto	Valor	
Coste operativo anual	4.376€	
Bolsa de ayudas accesibles	7.176€	
Punto de equilibrio	24 ventas/año	
ROI Año 1 (sin coste desarrollo)	+18% a +506%	Return on Investment
Viabilidad global	89% — PROYECTO VIABLE	

BLOQUE 4 — ARQUITECTURA GENERAL DEL SISTEMA

🕒 1.5 minutos · ~195 palabras · Pág. 6 del PDF · Módulo: DAW

(Pág. 6 del PDF — diagrama de arquitectura)

La arquitectura sigue un patrón en capas adaptado al paradigma reactivo de Reflex. Hay **cinco capas**:

1. **Navegador**: ejecuta React generado automáticamente. HTML/CSS/JS que yo nunca escribí.
2. **Páginas y componentes**: 30 archivos Python definiendo la UI.
3. **Estados**: 10 clases Python que almacenan datos y sincronizan la UI vía WebSocket.
4. **Servicios**: 4 módulos Python que encapsulan todas las operaciones con MongoDB.
5. **MongoDB Atlas**: base de datos NoSQL en la nube, 4 colecciones.

Las rutas dinámicas como `/courses/[course_id]` extraen el parámetro de la URL e inyectan su valor en el estado automáticamente.

Stack completo: **Python 3.14 + Reflex + MongoDB Atlas + Motor async + bcrypt + ~~Granian + Redis~~**.

1. bcrypt

¿Qué es? Es un algoritmo matemático diseñado específicamente para "hashear" (cifrar) contraseñas de forma segura.

BLOQUE 5 — LA BASE DE DATOS: MONGODB Y LOS MODELOS

🕒 1.5 minutos · ~195 palabras · Pág. 7 del PDF · Módulo: DAW

(Pág. 7 del PDF — modelo de datos)

¿Por qué MongoDB? Porque los cursos tienen estructura variable. Con SQL necesitaría nuevas tablas y JOINS costosos para cada variación. Con MongoDB, un curso es un **documento JSON flexible**.

El proyecto usa cuatro colecciones: `users`, `courses`, `contacts`, `categories`.

La colección `courses` aplica el patrón **embedding vs referencing**:

- **Lecciones y reseñas** embebidas: siempre se leen junto al curso.
- **Referencias a estudiantes e instructores**: solo ObjectIDs, porque se consultan independientemente.

Un detalle crítico: MongoDB almacena IDs como tipo `ObjectId`, no como string. Sin convertir con `str()`, la comparación `"abc123" == ObjectId("abc123")` devuelve `False` —un bug silencioso en producción.

Los servicios usan `async/await` con Motor: mientras MongoDB responde, el servidor atiende otras peticiones sin bloqueos.

BLOQUE 6 — AUTENTICACIÓN Y SISTEMA DE ROLES

🕒 2 minutos · ~260 palabras · Pág. 8 del PDF · Módulo: DAW

(Pág. 8 del PDF — AuthState y flujo de login)

Hay tres roles con capacidades distintas: **estudiante**, **instructor** y **administrador**.

El corazón es **AuthState**, con propiedades computadas mediante **@rx.var** que se recalculan automáticamente cuando el estado cambia:

```
class AuthState(rx.State):
    is_authenticated: bool = False
    current_user: dict = {}

    @rx.var
    def is_user_admin(self) -> bool:
        return self.current_user.get("role", "") == "admin"
```

El **flujo de login** en cinco pasos:

1. Buscar usuario por email —error genérico para no revelar qué campo es incorrecto.
2. Verificar contraseña con **bcrypt** (hash irreversible con salt aleatorio → bloquea ataques rainbow table).
3. Guardar usuario en estado: `self.current_user, self.is_authenticated = True`.
4. Redirigir según rol: `/admin/dashboard`, `/instructor/dashboard` o `/student/dashboard`.
5. El estado se propaga: navbar, menús y rutas protegidas se actualizan solos.

La **protección de rutas** usa Higher Order Components con **rx.cond()**. La protección real del backend está en los servicios: si el estado no lo permite, las operaciones de MongoDB no se ejecutan.

BLOQUE 7 — LOS ESTADOS DE REFLEX: EL BACKEND

🕒 2 minutos · ~260 palabras · Pág. 9 del PDF · Módulo: DAW

(Pág. 9 del PDF — jerarquía de estados y vars computadas)

El **State** es el concepto central de Reflex: una clase Python que almacena datos, define acciones y se sincroniza con el frontend vía WebSocket. Los diez estados forman una jerarquía con **AuthState** por encima de todos; los estados hijos heredan **current_user** sin duplicar código.

Búsqueda en tiempo real con **@rx.var**:

```
@rx.var
def filtered_courses(self) -> list[dict]:
    query = self.search_query.lower()
    return [c for c in self.courses
            if query in c.get("title", "").lower()]
```

El usuario escribe → el var se recalcula → la lista se actualiza. Sin llamadas al servidor.

Lo más poderoso de Reflex es el **ciclo completo automatizado**: el desarrollador no gestiona ninguna de estas capas manualmente.

1. El usuario pulsa un botón en el navegador.
2. Reflex captura el evento y llama al **event handler en Python**.
3. El handler muta el estado: **self.courses = await course_service.get_all()**.
4. Reflex detecta el cambio y envía el delta por **WebSocket** al frontend.
5. React re-renderiza solo los componentes afectados.

Sin fetch manual. Sin JSON. Sin Redux. Sin gestión de WebSocket. Todo ese código —que en un stack tradicional escribirías tú— lo genera y ejecuta Reflex por debajo.

BLOQUE 8 — PÁGINAS, COMPONENTES Y FUNCIONALIDADES

🕒 1 minuto · ~130 palabras · Pág. 10 del PDF · Módulo: DAW

(Pág. 10 del PDF — mapa de funcionalidades)

Cinco funcionalidades clave que demuestran la amplitud del proyecto —y que veremos en vivo a continuación:

1. **Visor de cursos:** sidebar con índice, progreso guardado por lección en MongoDB.
2. **Dashboards diferenciados:** admin, instructor y estudiante con datos reales de MongoDB.
3. **Búsqueda en tiempo real:** var computada sin llamadas al servidor.
4. **CRUD de cursos:** formulario unificado para crear y editar, con `$addToSet` para evitar duplicados atómicamente.
5. **Diseño responsive:** `rx.breakpoints()` genera media queries en una sola línea de Python.

[Role-Based Access Control](#)

BLOQUE 9 — CONCLUSIONES, AYUDAS Y PRÓXIMOS PASOS

🕒 1 minuto · ~130 palabras · Pág. 11–12 del PDF · Módulo: FOL + DAW

(Pág. 11 del PDF — ayudas disponibles)

Ayudas reales para el lanzamiento: tarifa plana SS (2.676€), Kit Digital (1.500€), Almería Emprende (3.000€).
Total: **7.176€** —prácticamente todos los costes operativos del primer año cubiertos.

(Pág. 12 del PDF — tabla resumen)

Dimensión	Resultado	
Técnica	Python full-stack, 0 JS manual	
Datos	MongoDB, embedding + referencing	
Seguridad	bcrypt, RBAC, RGPD, LSSI	Role-Based Access Control
Negocio	Equilibrio en 24 ventas/año	Reglamento General de Protección de Datos
Deploy	Reflex Cloud — en producción	Ley de Servicios de la Sociedad de la Información
Viabilidad global	89% — ALTAMENTE VIABLE	

La aplicación está desplegada y accesible ahora mismo en: **<https://e-learning-jcb-reflex-gray-orca.reflex.run/>**

Próximos pasos: Stripe, notificaciones WebSocket, exámenes/certificados, analítica con IA.

Python es suficiente para construir una plataforma web profesional, segura y comercialmente viable.

A continuación, la demostración en vivo.

APÉNDICE — RESPUESTAS A PREGUNTAS FRECUENTES

¿Por qué autónomo y no Sociedad Limitada? Para el primer año, el autónomo es más ágil y barato. La tarifa plana reduce la SS a 80€/mes. Si los beneficios superan 40.000€, el cambio a SL es más eficiente fiscalmente.

¿Qué pasa con el RGPD y los datos de los estudiantes? Los datos se almacenan en MongoDB Atlas con cifrado AES-256 en reposo y TLS en tránsito. La plataforma incluye política de privacidad, gestión de cookies y formularios con consentimiento explícito.

¿No es más lento que usar React directamente? En producción, Reflex compila a React optimizado. La diferencia de rendimiento es mínima para este tipo de aplicación. La ganancia en velocidad de desarrollo es enorme.

¿Se puede usar SQL en lugar de MongoDB? Sí. Reflex funciona con cualquier base de datos. MongoDB se eligió por la flexibilidad de esquema para cursos con estructura variable.

¿Cómo se hace el deploy? Reflex Cloud. La app está desplegada en: <https://e-learning-jcb-reflex-gray-orca.reflex.run/>

¿Qué fue lo más difícil de desarrollar? El sistema de estados con herencia y el bug del doble `on_mount`. Entender cómo el estado reactivo se sincroniza con el frontend requiere cambiar la forma de pensar respecto al desarrollo web tradicional.

Guion oral — Exposición teórica 15 minutos — E-Learning JCB ~1.950 palabras · ~130 palabras/minuto · Módulos: DAW · FOL · EMPRESA